PlayCarys Workflows

Operational reference — playcarys with flow corrections

Covers: first-time setup, launching, building, verifying middleware, reading logs, stopping servers, and packaging a clean zip to share.

Project layout

- back-end/ — NestJS API, port 3000
- front-end/ — static site + proxy server (dev-server.js), port 8080
- run.bat — one-click launcher for both, plus a public tunnel
- logs/ — dated access/error logs written by the middleware
- docs/ — MIDDLEWARE.md, this file, and other reference docs

1. Prerequisites

Needed once per machine:

- Node.js (LTS) and npm
- A terminal — PowerShell or Git Bash

> *Nothing else. No Python, no global CLI installs — the project is self-contained once dependencies are installed.*

2. First-time setup

Run once after cloning/unzipping the project, or whenever package.json changes:

```
cd back-end
npm install
npm run build

cd ../front-end
npm install
```

> *npm install regenerates node_modules/ (deliberately excluded from any shared zip — it is large and fully reproducible from package.json).*

3. Full launch — run.bat

The one-click path. Double-click run.bat from the project root, or run it from a terminal:

```
run.bat
```

What it does, in order:

- Builds the backend once if dist/main.js is missing
- Starts the backend (node dist/main.js) hidden, logging to logs/backend.log
- Starts the frontend (node dev-server.js) hidden, logging to logs/frontend.log
- Waits for the API to answer on :3000
- Starts a public localtunnel link (logs/tunnel.log) so other people can reach it
- Prints Local / Wi-Fi / Public URLs and opens http://localhost:8080
- Closing the window stops everything (taskkill on node.exe)

> *Use this when you want the full experience — LAN play, a public link for testing on another device. It is slower to start than the manual path below because of the tunnel step.*

4. Manual dev launch (two terminals)

Faster for iterating on code — no tunnel, no wait, and you see each server's own output live.

Terminal 1 — backend:

```
cd back-end
node dist/main.js
# or, to auto-rebuild on change:
npm run start:dev
```

Terminal 2 — frontend:

```
cd front-end
node dev-server.js
```

Then open:

```
http://localhost:8080
```

> *dev-server.js proxies /api and /socket.io through to the backend on :3000 automatically — the frontend never needs its own copy of API logic.*

5. Build the backend

```
cd back-end
npm run build
```

This runs nest build (TypeScript compiler) and writes back-end/dist/. A clean run prints nothing but the two lines above — any TypeScript error appears here, not at runtime.

> *dist/ is disposable — safe to delete anytime, npm run build regenerates it from src/. Never edit files inside dist/ directly.*

6. Verify the middleware

With the backend running (Section 4), from another terminal:

Logging

```
curl http://localhost:3000/api/v1/games
```

Check logs/access-YYYY-MM-DD.log (UTC date) grew a line ending "200 <n>ms - <ip>".

Error handling

```
curl -i http://localhost:3000/api/v1/does-not-exist
```

Response body is exactly: {"statusCode":404,"message":"...","timestamp":"...","path":"..."} — no stack trace. The full stack appears only in logs/error-YYYY-MM-DD.log.

Security headers + rate limit

```
curl -i http://localhost:3000/api/v1/games | findstr /i "x-frame x-content-type"

for /l %i in (1,1,12) do curl -s -o nul -w "%%i\n" -X POST ^
  http://localhost:3000/api/v1/auth/login -H "Content-Type: application/json" ^
  -d "{\"email\":\"x@x.com\",\"password\":\"wrongpassword\"}"
```

Attempts 11 and 12 should return 429. A parallel curl to /games should stay unaffected — the limiter is scoped to auth/upload routes only.

Router-level audit scoping

```
curl http://localhost:3000/api/v1/games
curl http://localhost:3000/api/v1/users
```

grep AUDIT logs\access-YYYY-MM-DD.log — only the /users call should produce an [AUDIT] line. /games never does.

File upload

Use Swagger at http://localhost:3000/api/docs, or curl multipart, on POST /api/v1/games/:id/bundle with a real publisher_id and a .zip under 50 MB.

7. Reading the logs

Everything lands in the project-root logs/ folder:

- logs/backend.log, frontend.log, tunnel.log — raw process stdout/stderr, written by run.bat via hidden.vbs
- logs/access-YYYY-MM-DD.log — one line per request (method, path, status, duration, IP), plus [AUDIT] lines for staff routes
- logs/error-YYYY-MM-DD.log — one detailed entry per thrown exception, including the stack trace

> *Dates are UTC, not local time — a file dated "today" in UTC may already say "tomorrow" on the clock late at night in IST. Not a bug.*

Both dated logs are append-only and flush every 10 seconds, or immediately on Ctrl+C (SIGINT/SIGTERM) — no entries are lost on a normal shutdown.

8. Stopping the servers

Launched via run.bat: close its terminal window — it runs taskkill /f /im node.exe on exit, which stops everything.

Launched manually: Ctrl+C in each terminal, or:

```
taskkill /f /im node.exe
```

> *This kills every node.exe process on the machine, not just PlayCarys' — fine on a dev machine, worth knowing if something else is also running under Node.*

9. Packaging a clean zip to share

Do NOT zip the project folder as-is — it is well over 100 MB and can leak real data. Exclude:

- node_modules/ (both folders) — regenerate with npm install
- back-end/dist/ — regenerate with npm run build
- front-end/games/ — extracted game bundles, the single largest contributor (100MB+)
- live_database.json and every live_database.json.bak-* — contains PLAINTEXT PASSWORDS for real accounts
- back-end/.auth-secret — the token-signing key; it auto-regenerates on first boot, never share it
- logs/, Videos/, .git/, public_url.txt — runtime noise

> *The app already ships safe built-in seed data (store.ts on the backend, db.js on the frontend) and starts fine with none of the above present. A friend gets a working project after npm install in both folders.*

A one-line PowerShell approach:

```
robocopy "playcarys with flow corrections" "_stage\playcarys with flow
corrections" /E ^
  /XD node_modules dist "dist (1)" games logs .git Videos ^
  /XF ".auth-secret" "live_database*" "public_url.txt"

Compress-Archive -Path "_stage\playcarys with flow corrections" -DestinationPath
playcarys.zip
```